

Signals

A Signal is a wrapper around a value that **knows when it changes**, and can tell Angular exactly who is using it — so only the UI that actually depends on it updates. This guide covers the full API, how Signals compare to plain variables and RxJS, and how to use them inside components and services.

1 · WHY SIGNALS EXIST

Before Signals: Angular had to guess

A plain `count = 0` is just a number — nothing reactive about it by itself. When `increment()` runs, Angular does **not** automatically know that `count` specifically changed. Historically Angular used **Zone.js**, a library that patches async browser APIs (`setTimeout`, click events, HTTP callbacks, promises) so Angular is notified whenever any of them runs — and on that notification, Angular walks the **entire** component tree top to bottom, checking every binding for changes, even ones that didn't change. It works, but it can do more work than strictly necessary.

Signals were introduced to let Angular know **exactly** which piece of the UI depends on **exactly** which piece of data — no more, no less.

2 · WHAT IS A SIGNAL, FROM ZERO

`count` vs. `count()` — do not mix these up

`count` — the Signal itself

```
count = signal(0);

console.log(count);
// -> Signal function reference,
// NOT the number
```

`count()` — the current value

```
console.log(count());
// -> 0

count.set(10);
count.update(v => v + 1);
```

"Reactive" means: things that depend on a Signal update automatically and precisely when that Signal's value changes — no manual re-checking needed. The difference from a normal variable: a plain `count = 0` cannot notify anyone when it changes; a Signal can, because reading/writing it goes through special functions (`count()`, `.set()`, `.update()`). In the template we write `{{ count() }}`, not `{{ count }}` — calling it also registers the template as a "dependent," so it updates automatically later.

3 · COMPUTED()

Derived, read-only state

```
count = signal(10);
doubleCount = computed(() => count() * 2);

console.log(doubleCount()); // 20
count.set(20);
console.log(doubleCount()); // 40 — recalculated automatically
```

`computed()` automatically tracks any Signal read inside its function — here, it noticed `count()` was read, so it knows to re-run whenever `count` changes. You never manually call `doubleCount.set(...)` — it derives itself, which is why it's **read-only**: there's no `.set()` or `.update()` on a computed Signal. Use it whenever a value is 100% derived from other Signals: totals, filtered lists, formatted strings, validation flags.

4 · EFFECT()

Running side effects when Signals change

```
effect(() => {
  console.log('Count changed to:', count());
});

// also common: syncing with localStorage
effect(() => {
  localStorage.setItem('cart-count', String(cartCount()));
});
```

`effect()` runs a **side effect** (logging, `localStorage`, calling a non-Angular library) — it does not return a usable value. It automatically tracks whichever Signals are read inside it, and re-runs whenever any of them change — same dependency-tracking mechanism as `computed()`. Do **not** use `effect()` to derive a value you'll use elsewhere — that's what `computed()` is for. And avoid writing to a Signal inside its own effect without care, to prevent update loops.

5 · SIGNALS IN THE TEMPLATE

From a click to a precise UI update

```
<p>{{ count() }}</p>
<button (click)="increment()">Increment</button>
```

User clicks

button



increment()

count.update(...)



Signal changes

notifies readers



Only that UI

updates

6 · SIGNALS + HTTPCLIENT

A realistic component: loading, error, success

```
export class ProductsComponent {
  products = signal<Product[]>([]);
  isLoading = signal(false);
  error = signal('');
  private productService = inject(ProductService);

  ngOnInit() {
    this.isLoading.set(true);
    this.productService.getProducts().subscribe({
      next: (res) => { this.products.set(res.products); this.isLoading.set(false); },
      error: () => { this.error.set('Failed to load.');
```

Service

HttpClient



API

Observable



Signal

.set(...)



UI

@for products()

7 · SIGNAL INPUTS & OUTPUTS

Modern component APIs, built on Signals

Traditional decorators

```
export class ProductCardComponent {
  @Input() product!: Product;
  @Output() addToCart =
    new EventEmitter<Product>();

  onAdd() {
    this.addToCart.emit(this.product);
  }
}
```

Signal-based

```
export class ProductCardComponent {
  product = input.required<Product>();
  addToCart = output<Product>();

  onAdd() {
    this.addToCart.emit(this.product());
  }
}
```

`input()` integrates directly with `computed()/effect()` dependency tracking — read it like any Signal, `product()` — and `input.required<T>()` enforces the input at compile time. Both styles work today; Signal Inputs are the modern direction.

8 · SIGNALS INSIDE SERVICES

A lightweight, Signal-based store

```
@Injectable({ providedIn: 'root' })
export class CartService {
  private _items = signal<Product[]>([]);

  // consumers can READ but not directly mutate
  items = this._items.asReadonly();
  itemCount = computed(() => this._items().length);

  addItem(product: Product) {
    this._items.update(list => [...list, product]);
  }
}
```

The private `_items` Signal can only be modified from **inside** the service, through controlled methods like `addItem()`. Components get `items` via `asReadonly()` — they can read and react to it, but cannot call `.set()` or `.update()` on it directly. This prevents random, hard-to-trace mutations from anywhere in the app — all changes flow through the service's explicit API. This pattern turns a service into a lightweight, Signal-based "store" for shared application state, no external state-management library required for many apps.

9 · SIGNALS VS. NORMAL VARIABLES

An honest, balanced note

Normal variables are **not** "bad" — for something purely internal that the template never reads reactively, a plain field is perfectly fine. Signals shine specifically when a value drives the UI, or when other calculations depend on it and must stay in sync automatically. Modern Angular is moving toward Signals as the primary reactivity model, but understanding the traditional model still helps you work in any Angular codebase, old or new.

Not a replacement — a complement

SIGNALS — GREAT FOR STATE	OBSERVABLES (RXJS) — GREAT FOR STREAMS
Local / component state	Async event streams (HTTP, WebSockets)
Application-wide shared state	Complex operator pipelines (debounce, switchMap, retry)
Derived values via <code>computed()</code>	HttpClient still returns Observables — that isn't changing
Simple, synchronous reads with <code>count()</code>	Multiple values over time, cancellation

```
import { toSignal, toObservable } from '@angular/core/rxjs-interop';

// Observable -> Signal (great for consuming HTTP data as a Signal)
products = toSignal(this.productService.getProducts(), { initialValue: [] });

// Signal -> Observable (great for feeding a Signal into an RxJS pipeline)
search = signal('');
search$ = toObservable(this.search);
```

Convert at the boundary with `toSignal` / `toObservable` whenever the two worlds need to meet.